Agentic Coding for Economists

Day 3: Orchestration, Autonomous Agents, Replication

Participant repo: `https://github.com/meleantonio/AC4E_Pavia`

Antonio Mele
University of Pavia

June 22–24, 2026

Day 2 Recap

What we built:

- Research design memo: question, literature, inputs, method, assumptions, limitations
- SDD chain: intent $\rightarrow$ requirements $\rightarrow$ design $\rightarrow$ tasks
- Literature map plus data or model-input map
- Prioritized issues with acceptance criteria and verification evidence

Context discipline: tight, wide, and external context recorded in `notes/context_patterns.md`.

Day 3 Goals

1. **Skills and review roles**: reuse procedures and separate implementation from review
2. **Subagents and MCP**: specialized contexts and structured external access where they add value
3. **Hooks and verification loops**: postcondition checks and execute–evaluate–revise until criteria are green
4. **Goals and plugins**: verifiable task files for Codex/Claude Code; inspect packaged skills and rules
5. **Cloud agents and swarms**: branch-isolated work with issues, labels, handoffs, and review-before-merge
6. **Autonomous-agent risk**: evaluate read/write boundaries, memory, execution, approvals, and evidence
7. **Final integration**: replication README, 5–7 minute presentation, and 30-day adoption plan

Definition of Done (Day 3): reviewable package, not a chat transcript.

Agent Harness: Three Tool Lanes

Participant repo ships the same economics workflows in three formats:

Piece	Cursor	Codex	Claude Code
Skills	`.cursor/skills/`	`.agents/skills/`	`.claude/skills/`
Subagents	`.cursor/agents/*.md`	`.codex/agents/*.toml`	`.claude/agents/*.md`
Hooks	`.cursor/hooks.json`	`.codex/hooks.json`	`.claude/settings.json`
MCP	`.cursor/mcp.json`	`.codex/config.toml`	`.mcp.json`

Portable copies: `agent-harness/cursor/`, `codex/`, `claude/`.

Verify UI labels, hook event names, and trust flows in your installed version.

Why Advanced Workflows Come Later

Before adding more autonomy

Skills, subagents, MCP, cloud agents, and swarms are useful only after the repository has scope, boundaries, and review criteria.

- **Needed first**: repo, project instructions, privacy boundaries, project brief, design memo, review criteria.
- **Without those**: advanced agents only automate confusion.
- **Today**: use skills, subagents, hooks, loops, and swarms to make review sharper, not to bypass judgment.

What is MCP?

MCP = Model Context Protocol

- A standard for connecting AI models to **external tools** (APIs, databases, filesystems)
- Supported agents invoke MCP servers; the AI then "sees" tools and resources they expose
- Use case for economists: pull FRED series, read data from controlled folders, search the web — all within a single chat
- **Local statistical software:** MCP can bridge to **Stata**, **MATLAB**, or **R** on your machine (via appropriate servers), so the agent can run batch jobs, scripts, or pull outputs in your environment—with your approval

MCP = bridges to real tools. The AI gains structured access instead of hallucinating data.

MCP Configuration

Locations:

- **Cursor:** project `.cursor/mcp.json` (example in `agent-harness/cursor/mcp/`)
- **Claude Code:** project `.mcp.json` (example in `agent-harness/claude/mcp/`)
- **Codex:** `.codex/config.toml` → `[mcp_servers.*]` (example in `agent-harness/codex/mcp/`)

Workshop server: `agent-harness/mcp/fred/` — search series, fetch metadata, pull observations.
Security: Never commit API keys. Use `env` / `env_vars` with `${FRED_API_KEY}`.

FRED MCP Example (Cursor)

```
{
  "mcpServers": {
    "fred": {
      "command": "python3",
      "args": ["-m", "fred_mcp_server"],
      "env": {
        "FRED_API_KEY": "${FRED_API_KEY}",
        "PYTHONPATH": "agent-harness/mcp/fred/src"
      }
    }
  }
}
```

Key from: `https://fredaccount.stlouisfed.org/apikeys` **Full setup:** `agent-harness/mcp/fred/README.md`

When MCP Adds Value vs Overkill

Adds value:

- Recurring data pulls (FRED, ECB, BIS)
- Scoped filesystem access (e.g. `data/` only)
- Web search for literature
- Local Stata/MATLAB/R workflows (MCP bridge to your machine)

Overkill:

- One-off data download (browser + save)
- Single analysis (manual run)
- Heavy custom tooling

Economists: FRED is the obvious win. Start with one MCP.

What Are Subagents?

Subagents are specialized AI assistants with their own context window.

- Each gets its own **prompt**, **tools**, and optionally a different **model**
- Use when you need: **context isolation**, **parallel work**, or **role specialization**
- Differ from main agent: focused on one job; from skills: separate context

Economics example: "Have pr-reviewer review the code changes and bibliographer check the references section" — two subagents in parallel.

Subagent / Custom Agent Format

Cursor and Claude Code use markdown with YAML frontmatter; **Codex** uses TOML:

Markdown (Cursor / Claude)

`name`, `description`
`model`: `fast` or `inherit`
`readonly`: `true` for reviewers
Body: role prompt and checklist

TOML (Codex)

`name`, `description`
`sandbox_mode = "read-only"`
`developer_instructions` = prompt
Path: `.codex/agents/*.toml`

Portable copies: `agent-harness/cursor/subagents/`, `codex/agents/`, `claude/agents/`.

Verify field names and UI labels in your tool version.

Subagent Example: PR Reviewer

```
---
name: pr-reviewer
description: PR review specialist for econometric code.
  Use when reviewing pull requests for consistency,
  statistical correctness, and style.
model: fast
readonly: true
---

You are a rigorous code reviewer for econometric projects.

When invoked on a PR:
1. **Consistency**: Project rules, naming, patterns
2. **Correctness**: Coefficients, SEs, missingness, units
3. **Style**: Docstrings, citations, README

Report: pass / needs changes + line-level suggestions.
```

Subagent Example: Codex PR Reviewer

```
name = "pr-reviewer"
description = "Reviews economics PRs for scope, reproducibility, and privacy."
sandbox_mode = "read-only"
developer_instructions = """
You are a rigorous reviewer for economics research repositories.
Return blockers first, then suggestions. Do not edit files.
"""
```

Same role, different file shape. Copy from `agent-harness/codex/agents/pr-reviewer.toml`.

Role Specialization (Card–Krueger Harness)

pr-reviewer Scope, privacy, reproducibility, test evidence on PRs.

data-reviewer Panel balance, variable coding, synthetic-data caveat.

literature-reviewer BibTeX accuracy, citation–claim matching, overclaiming.

loop-verifier Execute–evaluate–revise verdicts against acceptance criteria.

replicator Clean-run path, dependencies, hardcoded paths, output hash.

One clear role per subagent. Avoid generic "helper" agents.

When to Use Subagents vs Main Agent vs Skills

Use subagent	Multi-step review, long exploration, parallel streams
Use skill	Single-purpose quick task (e.g. generate changelog)
Use main agent	General orchestration, decisions, merging

Invoke explicitly: "Use the pr-reviewer subagent to review PR #42" or `/pr-reviewer review this PR`

What Are Skills?

Skills extend the agent with reusable workflows.

- **Day 2 recap:** you used the bundled `/sdd` skill; today you *author* your own
- Loaded when the **description** matches the user request
- Unlike subagents: **no separate context** — they augment the main agent
- Same pattern across tools: a discoverable `SKILL.md` with optional `references/`, `scripts/`, and `assets/`

Example trigger: "Verify that my pipeline in `scripts/run_analysis.py` is reproducible" → replication-checker skill loads

SKILL.md Structure

Anatomy:

- **Frontmatter:** `name`, `description` (required). Description is the primary trigger.
- **Body:** instructions, workflows, when to read `references/` or `scripts/`
- **Optional:** `references/`, `scripts/`, `assets/` for progressive disclosure

Progressive disclosure: Keep SKILL.md under ∼500 lines. Move detailed checklists to `references/`.

Skill-Creator Principles

1. **Concise is key:** Context window is shared. Only add what the agent doesn't already have.
2. **Degrees of freedom:** Low (scripts, strict steps) for fragile ops; high (text guidance) for variable decisions.
3. **Progressive disclosure:** SKILL.md stays lean; load `references/` or `scripts/` only when needed.

Validate-iterate: Use the skill on a real task. Fix what breaks.

Building the Replication-Checker Skill

```
---
name: replication-checker
description: Verifies that research pipelines run from clean
  state with no hardcoded paths, declared dependencies,
  and reproducible output.
---

# Replication Checker

When invoked:
  - Identify main entry point (script, notebook, Make)
  - Check for hardcoded paths (see references/)
  - Verify dependencies: requirements.txt
  - Run minimal/dry-run; report success or failure

Output: green/yellow/red, blockers, run command
```

Skill Taxonomy

Skill	Trigger phrases	Degree of freedom
replication-checker	verify replication, check pipeline	Medium
hooks	configure lifecycle hook, after edit	Low
loop-on-verification	iterate until criteria green	Medium
literature-mapper	standardize bib, map references	Medium
data-contract-enforcer	validate data, check units	Low (script-heavy)

Portable copies: `agent-harness/cursor/skills/`, `codex/skills/`, `claude/skills/`.

What Are Hooks?

Hooks are postcondition listeners: the tool runs a command when a lifecycle event fires.

- **After file edit:** run `pytest` when `did_analysis.py` changes
- **Session stop:** append a reminder to `notes/orchestration_log.md`
- **Before tool use:** block writes to raw-data folders (where supported)

Hooks catch breakage early. They do *not* replace human diff review.

Hook Configuration by Tool

Tool	Config path	Example events
Cursor	`.cursor/hooks.json`	`afterFileEdit`, `stop`
Claude Code	`.claude/settings.json`	`PostToolUse`, `Stop`
Codex	`.codex/hooks.json`	verify in your version

Worked example: `agent-harness/cursor/skills/hooks/SKILL.md` and `.cursor/hooks/economics-hooks-example.json`.

Card–Krueger hook fires `pytest examples/card-krueger-toy/tests` after edits to the DiD script.

Loop on Verification

Three phases, repeated until exit or escalation:

1. **Implement** — narrowly scoped change against acceptance criteria
2. **Evaluate** — `loop-verifier` subagent or replication-checker skill
3. **Revise** — address only listed blockers; record iteration in orchestration log

Verdicts: GREEN → human review; YELLOW → loop again; RED or 3 iterations → stop and open a new issue.

Loop vs Swarm

Verification loop

- One issue, one agent stream
- Sequential implement–evaluate cycles
- Exit when criteria are green
- Skill: `loop-on-verification`

Swarm

- Many issues in parallel waves
- GitHub labels and dependencies
- Merge after subagent review
- Example: `examples/card-krueger-swarm/`

Both require acceptance criteria first. Neither replaces human merge approval.

Goal Files and the `/goal` Command

A **goal** is persistent task state the agent retrieves across sessions — the local equivalent of a GitHub issue.

Field	Write verifiable criteria
`name`	one-line task identifier
`description`	economic output when done
`approach`	files to edit, method to use
`acceptance_criteria`	checkbox list; each item checkable by command
`constraints`	raw data untouched; synthetic caveat required

Worked examples: `examples/card-krueger-goals/goals/`.
Attach: Claude Code `/goal`; Codex Goals UI; Cursor Cloud Agent task description.

What Are Cloud / Remote Agents?

Cloud/remote agents run outside the local editor and return branch or PR-level work.

- They clone your repo, work on a branch, and may produce a PR.
- They are useful for long, isolated, reviewable tasks that do not need private data.
- They are a bad fit for live demos, vague research decisions, or confidential files.
- Cursor, Codex, Claude, and other tools expose this pattern differently; verify current UI and field names in official docs.

Handoff flow: Task $\rightarrow$ agent runs remotely $\rightarrow$ branch/PR created $\rightarrow$ **you review, then merge**

Branch Handoff Protocol

1. **Before launch:** Create or specify the branch; ensure it's based on current `main`
2. **Task description:** Clear scope, file paths, "work on branch X"
3. **During run:** Agent clones, checks out branch, executes. You keep working.
4. **On completion:** Agent pushes branch, opens PR
5. **Review:** Open PR, read diff, run tests. Use pr-reviewer subagent.
6. **Decision:** Merge, request changes, or close

Safety and Prompt Injection Awareness

Safety rules:

- **No merge without review** — agent output can be wrong or off-scope
- **Branch isolation** — work stays on branch until you merge
- **No secrets in prompts** — use env vars; exclude `.env` through privacy boundaries
- **Sanitize external input** — avoid piping raw user text into agent prompts

Prompt injection: Malicious content in inputs that steers the agent. Lower risk when you control inputs (your code).

Review-Before-Merge Protocol

Checklist before merging a remote-agent PR:

- Run tests
- Spot-check 3 random files
- Verify no logic changes unless requested
- No hardcoded paths or secrets in diff

Key prompt: "Generate docstrings for all Python files in `scripts/`. Use Google-style. Work on branch `agent/docstrings`. Do not modify function bodies."

Cursor Plugins (Distribution Layer)

A **plugin** packages skills, rules, agents, and commands behind a `plugin.json` manifest.

- **Use when:** sharing harness pieces across projects or a research team
- **Inspect first:** read `plugin.json` before installing on a research repo
- **Exercise 9:** name one declared skill or rule and how it would change the Card–Krueger workflow

Teaching point: plugins distribute the same `SKILL.md` anatomy you author today.
Codex and Claude Code have their own packaging paths — verify in your version.

Autonomous Agents: OpenClaw, Hermes, Eve

Agent	What to notice	Workshop stance
OpenClaw	Local-first assistant; channels, tools, skills, sandboxing, multi-agent routing	Good for personal-agent and channel-safety discussion; optional reading
Hermes	Nous Research agent focused on long-running context and user-specific capability growth	Useful for memory, persistence, and ownership of research context
Eve	Vercel framework for building agents with instructions, skills, tools, sandboxes, channels, subagents, schedules	Use only after checking current docs and access boundaries

Rule: More autonomy means more need for logs, review gates, and reproducibility evidence.

Autonomous-Agent Risk Card

Fill one risk card before letting any autonomous agent touch a research repo:

- **Read boundary:** what files, channels, or data can it inspect?
- **Write boundary:** what can it edit, trigger, schedule, or publish?
- **Memory:** what state persists across sessions, and who controls it?
- **Execution:** local, cloud, sandbox, or production system?
- **Review gate:** what human approval is required before outputs enter the repo?
- **Evidence:** what logs, diffs, commands, or replication outputs prove what happened?

Artifact: `agent-harness/autonomous_agent_risk_card.md`

Risk Card Decision

Approve only if you can answer:

- **Read**: what can it inspect?
- **Write**: what can it edit, trigger, publish, or schedule?
- **Memory**: what persists and who controls it?
- **Execution**: local, sandbox, cloud, production, or mixed?
- **Evidence**: which logs, diffs, transcripts, or outputs prove what happened?

Decision labels

- Safe for read-only use.
- Safe for branch-isolated write use.
- Not safe for this workshop repo.
- Needs more documentation before use.

Rule: do not connect private data unless every boundary and approval gate is acceptable.

Map New Agents Back to the Harness

Autonomous-agent feature	Course harness equivalent
Markdown instructions	`AGENTS.md`, `CLAUDE.md`, Cursor rules
Skills / tools	`SKILL.md`, MCP tools, validation scripts
Postconditions	Hooks, CI, verification commands
Task memory	Goal files, GitHub issues, orchestration log
Channels / schedules	GitHub issues, CI, scheduled checks
Subagents / workers	data-reviewer, literature-reviewer, loop-verifier
Sandbox	Branch, worktree, remote runner, protected main
Logs	PR description, orchestration log, replication README

Teaching point: The architecture transfers; the UI changes.

What Is a Swarm?

A **swarm** is **several agents coordinated** toward one project outcome — not a single prescribed toolchain.

- You might combine **foreground agent**, **subagents**, **skills**, and **remote agents** in one workflow
- Orchestration can be **human-led**, **scripted**, or **agent-assisted** (planner creates issues; implementers execute)
- The next slides use **GitHub + remote agents** as one practical pattern; adapt labels and tools to your lab

GitHub as the Control Plane

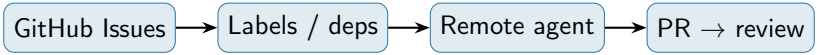
Issues = units of work. Make dependencies explicit:

- **Labels** (examples): `parallel`, `sequential`, `blocked-by:#12`, `ready`
- **Project columns** or milestones for waves: Wave A (parallel) → Wave B (after merge of #5)
- Planner (you or an agent) opens issues with **acceptance criteria**; sub-orchestrators triage and assign remote-agent runs

Rule of thumb: Independent tasks → parallel agents; dependent tasks → sequential launches after prior PR merges.

From Issues to PRs

As in the remote-agent section above: each issue can drive one **branch** and one agent run when the task is large enough to isolate.



Foreground agent on your machine is still valid for short edits; remote agents help when runs are long or you want many branches at once.

Layered Orchestration

Three layers (names are illustrative — tune to your repo):

Planner Creates or refines the issue graph (human or main Agent). Outputs: labeled issues, milestones.

Implementation swarm Remote agents (and humans) work issues in parallel or sequence per labels.

Review / consistency swarm Subagents + skills: `pr-reviewer`, `replicator`, replication-checker; optional "review lead" Agent pass across PRs

Sub-orchestrators can be **different chats** or **different remote-agent tasks** with narrow prompts (e.g. "only consistency of notation between `model/` and `paper/`").

Handoffs and Ledger (Still Essential)

Branches: e.g. `feat/hank-ss`, `feat/hank-irf` — one coherent branch per issue when using remote agents.

- Merge only after review; keep `main` stable

Handoff files: `notes/handoff_*.md` — column names, units, SS outputs IRFs consume
Orchestration log: `notes/orchestration_log.md` — Stream | Branch | Issue | Status | PR | Reviewer

Example: Card–Krueger Four-Stream Swarm

Template: `examples/card-krueger-swarm/` with issue files in `issues/`.

Stream	Label	Reviewer
A: Data cleaning	`parallel` / wave 1	pr-reviewer
B: Literature	`parallel` / wave 1	literature-reviewer
C: DiD estimation	`blocked-by:#A,#B`	data-reviewer
D: Pre-trends figure	`blocked-by:#C`	pr-reviewer

Macro analogue: parallel literature + data prep → sequential estimation → figures and write-up.

Example: Branches and Commands

Setup (illustrative):

```
git checkout main && git pull
git checkout -b feat/hank-ss
# ... launch remote agent on issue #1 for this branch; open PR when done.
```

After #1 merges: create `feat/hank-irf` from updated `main`; launch agent for issue #2.
Optional further reading (other orchestration metaphors):
`https://github.com/steveyegge/gastown`

Safety Rules: No Merge Without Review

Before merging:

1. Run **pr-reviewer subagent** on each PR
2. Run **replication-checker skill** on the combined pipeline
3. Merge in dependency order (e.g. data $\rightarrow$ equilibrium $\rightarrow$ IRFs $\rightarrow$ paper)

Swarm lab protocol: Phase 1 Setup (15 min) $\rightarrow$ Phase 2 Parallel (90 min) $\rightarrow$ Phase 3 Review & Merge (45 min) $\rightarrow$ Phase 4 Verification (15 min)

Definition of Done for Day 3

1. **Review role used**: subagent or saved role prompt reviews a real diff (e.g. data-reviewer on Card–Krueger).
2. **Reusable workflow used**: replication-checker, hooks skill, or loop-on-verification skill.
3. **Hook drafted**: one postcondition entry for your tool lane (Exercise 6).
4. **Verification loop logged**: at least one iteration with a GREEN/YELLOW/RED verdict in `notes/orchestration_log.md`.
5. **Goal file**: verifiable acceptance criteria for a Card–Krueger task (Exercise 8).
6. **Plugin inspected**: read one `plugin.json`; explain one skill or rule (Exercise 9).
7. **Risk card**: `agent-harness/autonomous_agent_risk_card.md`.
8. **Replication path**: `replication/README.md` plus clean-run evidence or documented blocker.
9. **Communication**: 5–7 minute presentation outline and 30-day adoption plan.

Day 3 Deliverables

Artifact	Location
Subagent(s)	`agent-harness/<tool>/` → your tool path
Skill(s)	replication-checker, hooks, loop-on-verification
Hook config	`.cursor/hooks.json` or tool equivalent
Goal file	`examples/card-krueger-goals/goals/`
Swarm plan	`examples/card-krueger-swarm/` + orchestration log
Risk card	`agent-harness/autonomous_agent_risk_card.md`
Replication README	`replication/README.md`
Presentation and plan	`notes/30_day_plan.md`

Final Integration Before Presentation

End today by making the project coherent:

- **Paths**: scripts, docs, outputs, and README agree.
- **Definitions**: variable names, units, primitives, parameters, and outputs are consistent.
- **Evidence**: output files exist or blockers are documented.
- **Claims**: presentation language matches the evidence and limitations.
- **Privacy**: boundaries are still respected.

Goal: a coherent, reviewable research package.

Replication README

A colleague should be able to follow it without guessing.

```
# Replication README

## Overview
## Software and versions
## Data or model inputs
## Setup
## Run instructions
## Expected outputs
## Troubleshooting
## Known limitations
## Contact or ownership
```

Use your lane: `Rscript`, `stata-mp -b do`, `matlab -batch`, `python`, `julia -project`, or the shortest honest manual sequence.

Final Presentation

5–7 minutes, not a full seminar:

1. **Research question**
2. **Why it matters**
3. **Data or model and method**
4. **Main result, prototype, or blocker**
5. **Agentic workflow**: what the agent did and what you reviewed
6. **Replication status**
7. **Next 30 days**

Avoid: "The AI proved the effect." Designs, assumptions, data, and review evidence support claims.

30-Day Adoption Plan

Week	Sustainable habit
1	Add or improve `AGENTS.md`; document privacy boundaries; define one verification check.
2	Choose one real task; create an issue; work on a branch; review the diff before merge.
3	Create one reusable skill or saved prompt and one reviewer role prompt; use them on a real artifact.
4	Run one replication check; update the README; share the workflow with a coauthor, RA, or colleague.

Habit to keep: before asking an agent to act, name the scope, context, forbidden areas, evidence, and human review.